

Analisi comparativa: igrab vs ihold

Equivalenza dell'incremento atomico di `i_count` nel contesto di `fserror_report`

Kernel Linux for-next ~7.0-rc1 · arch x86-64 · CONFIG_LOCKDEP=y · CONFIG_PREEMPT_DYNAMIC=y

1. Contesto

Il bug segnalato da syzbot (extid 5eb0d61dfb76ca12670c) produce un warning lockdep **{SOFTIRQ-ON-W} → {IN-SOFTIRQ-W}** su `i_lock_key` all'interno di `igrab()`, chiamata da `fserror_report()` nel contesto di un BLOCK softirq (`ksoftirqd/0`, PID 15). La correzione proposta sostituisce `igrab()` con `ihold()`. Questo documento dimostra, espandendo ricorsivamente le due funzioni fino all'istruzione assembly, che producono lo stesso effetto su `i_count` nel contesto in cui vengono usate, rendendo la sostituzione sicura e corretta.

2. igrab — percorso dell'incremento

Punto di partenza: `__iget(inode)`, chiamata dentro `igrab` dopo che il check su `I_FREEING` | `I_WILL_FREE` ha avuto esito negativo (inode vivo). Ogni funzione viene sostituita dal suo corpo, ricorsivamente.

2.1. Livello 1 — `__iget` · `fs/inode.c`

```
static inline void __iget(struct inode *inode)
{
    lockdep_assert_held(&inode->i_lock); /* assert debug, no-op */
    atomic_inc(&inode->i_count);         /* espanso al livello 2 */
}
```

lockdep_assert_held verifica a runtime che il lock sia tenuto — è un assert di debug che non modifica alcuno stato. L'unica operazione effettiva è `atomic_inc`.

2.2. Livello 2 — `atomic_inc` · `include/linux/atomic/atomic-instrumented.h`

```
static __always_inline void atomic_inc(atomic_t *v)
{
    instrument_atomic_read_write(v, sizeof(*v)); /* espanso sotto */
    raw_atomic_inc(v);                             /* espanso al livello 3 */
}

/* instrument_atomic_read_write si espande in: */
static __always_inline void
instrument_atomic_read_write(const volatile void *v, size_t size)
{
    kasan_check_write(v, size); /* checker KASAN */
    kcsan_check_atomic_read_write(v, size); /* checker KCSAN */
    instrument_atomic_check_alignment(v, size); /* check allineamento */
}
```

kasan_check_write, kcsan_check_atomic_read_write e instrument_atomic_check_alignment sono tutti checker di sanitizer (KASAN, KCSAN): non modificano i_count, non cambiano lo stato del sistema in produzione. L'operazione reale è raw_atomic_inc.

2.3. Livello 3 — raw_atomic_inc • include/linux/atomic/atomic-arch-fallback.h

```
static __always_inline void raw_atomic_inc(atomic_t *v)
{
    #if defined(arch_atomic_inc)
        arch_atomic_inc(v); /* su x86: NON definita → si cade nel #else */
    #else
        raw_atomic_add(1, v); /* percorso effettivamente preso su x86 */
    #endif
}

/* raw_atomic_add si espande a: */
static __always_inline void raw_atomic_add(int i, atomic_t *v)
{
    arch_atomic_add(i, v); /* espanso al livello 4 */
}
```

Su x86-64 arch_atomic_inc non è definita come macro separata, quindi raw_atomic_inc cade nel ramo #else e chiama raw_atomic_add(1, v), che a sua volta chiama arch_atomic_add. Nessun effetto su i_count ancora: si attraversano i wrapper.

2.4. Livello 4 — arch_atomic_add • arch/x86/include/asm/atomic.h

```
static __always_inline void arch_atomic_add(int i, atomic_t *v)
{
    asm_inline volatile(LOCK_PREFIX "addl %1, %0"
                        : "+m" (v->counter)
                        : "ir" (i) : "memory");
}
```

Istruzione assembly terminale di igrab:

```
lock addl $1, i_count(%rdi)
```

*LOCK ADDL: istruzione x86 che incrementa atomicamente il valore in memoria di 1. Il prefisso LOCK garantisce atomicità su SMP: nessun altro core può leggere o scrivere i_count tra la lettura e la scrittura. **Effetto definitivo:** i_count += 1. La funzione non ritorna alcun valore.*

3. ihold — percorso dell'incremento

Punto di partenza: `atomic_inc_return(&inode->i_count)`, unica operazione effettiva dentro `ihold`. Il `WARN_ON` usa il valore di ritorno solo per un controllo di sanità e non tocca più `i_count`.

3.1. Livello 0 — `ihold` • `include/linux/fs.h`

```
void ihold(struct inode *inode)
{
    WARN_ON(atomic_inc_return(&inode->i_count) < 2);
    /* WARN_ON non modifica i_count: usa solo il valore ritornato */
}
```

3.2. Livello 1 — `atomic_inc_return` • `include/linux/atomic/atomic-instrumented.h`

```
static __always_inline int atomic_inc_return(atomic_t *v)
{
    kcsan_mb(); /* memory barrier KCSAN, no-op */
    instrument_atomic_read_write(v, sizeof(*v)); /* stessi checker di prima */
    return raw_atomic_inc_return(v); /* espanso al livello 2 */
}
```

*kcsan_mb() e instrument_atomic_read_write sono identici a quelli visti in igrab: checker di sanitizer, nessun effetto su `i_count`. La differenza rispetto ad `atomic_inc` è che questa funzione **ritorna** il nuovo valore di `i_count`, necessario per il `WARN_ON`.*

3.3. Livello 2 — `raw_atomic_inc_return` • `atomic-arch-fallback.h`

```
static __always_inline int raw_atomic_inc_return(atomic_t *v)
{
    #if defined(arch_atomic_inc_return)
        return arch_atomic_inc_return(v);
        /* su x86: NON definita → ramo NON preso
           verificato: grep "arch_atomic_inc_return" arch/x86/.../atomic.h → nessun risultato */
    #elif defined(arch_atomic_inc_return_relaxed)
        int ret;
        __atomic_pre_full_fence();
        ret = arch_atomic_inc_return_relaxed(v);
        __atomic_post_full_fence();
        return ret;
        /* su x86: NON definita → ramo NON preso
           verificato: grep "arch_atomic_inc_return_relaxed" arch/x86/.../atomic.h → nessun risultato */
    #else
        return raw_atomic_add_return(1, v); /* ← ramo effettivamente preso su x86 */
    #endif
}
```

Verificato sul kernel reale (grep su `arch/x86/include/asm/atomic*.h`): né `arch_atomic_inc_return` né `arch_atomic_inc_return_relaxed` sono definite su x86. Si cade quindi nel ramo `#else`: `raw_atomic_add_return(1, v)`. Questo rende il percorso di `ihold` simmetrico a quello di `igrab`: entrambi passano per una funzione `raw_atomic_add*` che chiama la corrispondente `arch_atomic_add*`.

3.4. Livello 3 — raw_atomic_add_return • atomic-arch-fallback.h

```
static __always_inline int raw_atomic_add_return(int i, atomic_t *v)
{
    #if defined(arch_atomic_add_return)
        return arch_atomic_add_return(i, v); /* su x86: DEFINITA → ramo preso
        verificato: riga 83 + #define arch_atomic_add_return (riga 87) */
    #elif defined(arch_atomic_add_return_relaxed)
        /* ... */
    #else
        /* ... */
    #endif
}
```

arch_atomic_add_return è definita su x86 sia come funzione (riga 83) sia come macro #define arch_atomic_add_return arch_atomic_add_return (riga 87) — è questa macro che rende defined(arch_atomic_add_return) vero nel preprocessore. Si prende il primo ramo #if.

3.5. Livello 4 — arch_atomic_add_return • arch/x86/include/asm/atomic.h

```
static __always_inline int arch_atomic_add_return(int i, atomic_t *v)
{
    return i + xadd(&v->counter, i);
}
```

3.6. Livello 5 — espansione di xadd • arch/x86/include/asm/cmpxchg.h

xadd è una catena di tre macro. Mostriamo prima le definizioni, poi la sostituzione con i valori reali del nostro caso.

Definizioni:

```
#define xadd(ptr, inc)      __xadd((ptr), (inc), LOCK_PREFIX)
#define __xadd(ptr, inc, lock) __xchg_op((ptr), (inc), xadd, lock)
```

__xchg_op è una macro generica (usata anche da xchg e cmpxchg) che seleziona il suffisso dell'istruzione in base a sizeof(*ptr):

```
#define __X86_CASE_B 1 /* byte */
#define __X86_CASE_W 2 /* word */
#define __X86_CASE_L 4 /* long (int, 32 bit) */
#define __X86_CASE_Q 8 /* quad (long, 64 bit) */
```

Sostituzione con i valori del nostro caso:

Nel nostro caso: ptr = &v->counter (tipo int*), inc = 1, op = xadd, lock = "lock ". Poiché sizeof(int) = 4, si entra nel caso __X86_CASE_L. Eliminando i casi non presi, la macro si riduce a:

```
{
    int __ret = 1; /* arg = 1 */
    /* sizeof(int) == 4 → caso __X86_CASE_L */
    asm_inline volatile("lock " "xadd" "l %0, %1"
        : "+r" (__ret), /* reg in/out: parte a 1 */
        "+m" (v->counter) /* memoria: i_count */
        : : "memory", "cc");
    /* dopo l'istruzione: */
    /* __ret = vecchio valore di i_count */
    /* v->counter = vecchio + 1 */
    __ret; /* valore di ritorno */
})
/* arch_atomic_add_return restituisce: 1 + __ret = nuovo i_count */
```

Istruzione assembly terminale di `ihold`:

```
lock xaddl  %eax, i_count(%rdi)
; prima: %eax = 1, i_count = N
; dopo:  %eax = N, i_count = N+1  (exchange-and-add)
; arch_atomic_add_return calcola 1 + N = N+1  (per il WARN_ON)
```

*LOCK XADDL (exchange-and-add): in un'unica operazione atomica, scrive il nuovo valore (`i_count + 1`) in memoria **e** salva il vecchio valore di `i_count` nel registro. Il vecchio valore serve solo per calcolare il valore di ritorno di `atomic_inc_return`, che a sua volta serve solo al `WARN_ON`. **Effetto definitivo su `i_count`: `i_count += 1`. Identico al `LOCK ADDL` di `igrab`.***

4. Confronto: le istruzioni terminali

Funzione	Percorso	Istruzione terminale	Effetto su i_count
igrab	__iget → atomic_inc → raw_atomic_inc → raw_atomic_add(1, v) [#else] → arch_atomic_add	lock addl \$1, i_count	i_count += 1
ihold	atomic_inc_return → raw_atomic_inc_return → raw_atomic_add_return(1,v) [#else] → arch_atomic_add_return → xadd → __xadd → __xchg_op	lock xaddl \$1, i_count add \$1, %eax	i_count += 1

ADDL e XADDL producono lo stesso risultato su i_count. XADDL è «exchange-and-add»: atomicamente somma il valore e restituisce il vecchio contenuto nel registro. ADDL somma senza restituire nulla. ihold usa XADDL perché atomic_inc_return deve ritornare il nuovo valore per il WARN_ON(result < 2). Ma sul contatore i_count, entrambe le istruzioni scrivono esattamente lo stesso risultato: il valore precedente più 1.

5. Cosa cambia davvero tra igrab e ihold

Operazione	igrab	ihold	Effetto permanente?
preempt_disable / enable	Sì (si annulla)	No	No — torna al valore originale
spin_lock / spin_unlock	Sì (si annulla)	No	No — lock rilasciato prima del return
Interazione con lockdep	Sì → può scattare WARNING	No	Solo nel checker, nessun effetto runtime
Lettura di i_state	Sì	No	No — check sempre falso se i_count >= 1
Incremento di i_count	lock addl \$1	lock xaddl \$1	Sì — identico in entrambe
WARN_ON(result < 2)	No	Sì	No — branch mai preso se i_count >= 1

6. Conclusione

Al termine di entrambe le chiamate, l'unico stato permanentemente modificato è lo stesso: inode->i_count è incrementato di 1.

Tutto ciò che igrab fa in più — spin_lock, check i_state, spin_unlock, interazione con lockdep — produce effetti **temporanei** che si annullano prima del return, oppure effetti limitati al checker lockdep che non influenzano il comportamento a runtime.

Il check su `I_FREEING | I_WILL_FREE` è l'unica logica aggiuntiva semanticamente rilevante di `igrab`. Nel contesto di `ferror_report` il caller tiene già una reference (`i_count >= 1`): poiché `I_FREEING` viene settato **solo** in `iput_final()` quando `i_count` scende a zero, la condizione è invariabilmente falsa e il ramo `return NULL` non viene mai preso.

La sostituzione `igrab` → `ihold` è quindi equivalente e corretta.